Relatório Técnico — Conformidade entre documento de requisitos ERL e microsserviços gerados por LLM

Disciplina: AED — Processo e Artefato
Grupo: Pedro Blamires Cordeiro, Adriano Faleiro Gregório, Gabriel Borges, Heitor Felipe Trindade Deodato, Jefferson Felex de Faria, Vinícius Rodrigues Gonçalves Ferreira, Gabriel Rodrigues do Vale
Data: 2026-10-21

Resumo

Propomos o processo ERL (Especificação de Requisitos para LLM) e um modelo de documento de requisitos em Markdown para sistemas de **microsserviços**, com arquitetura, contrato de rotas e eventos, convenções transversais, regras atômicas, ordem de validação e critérios de aceitação concretos. O objeto do experimento é um módulo de **logística de entregas** de um marketplace, com cinco microsserviços (Transportadoras, Cotação, Envios, Rastreamento e Notificações), comunicação por HTTP e por eventos, e escolha da transportadora pelo cliente. A conformidade foi medida como a fração dos 77 critérios de aceitação aprovados por um oráculo que sobe os serviços como processos separados e os testa pela rede. Em 30 gerações independentes com Claude Opus 5.5 a partir do documento ERL, **todas atingiram 100% de conformidade** (77/77), e a hipótese nula de conformidade $\leq 85\%$ foi rejeitada (Wilcoxon, $p = 2{,}2 \times 10^{-8}$). Uma especificação narrativa com o mesmo conteúdo também atingiu 100% em 30 de 30 execuções, repetindo o efeito teto de um estudo preliminar com um sistema menor.

---

1. Introdução e problema

LLMs geram código a partir de texto, mas não pedem esclarecimentos: diante de uma ambiguidade, escolhem uma interpretação. Em microsserviços o risco é maior, porque cada serviço pode estar "certo" isoladamente e o sistema estar errado: basta que dois serviços entendam o mesmo contrato de formas diferentes. O problema investigado é: **um documento de requisitos estruturado consegue levar uma LLM a produzir um sistema de microsserviços com conformidade de pelo menos 85% em relação aos requisitos?**

O trabalho seguiu um ciclo iterativo em dois estudos:

	Estudo preliminar	Estudo principal
Sistema	Biblioteca (um módulo Python)	Logística de entregas (5 microsserviços FastAPI)
Critérios de aceitação	64	77
Interface testada	Chamadas de função	HTTP entre processos e eventos
Resultado	100% nas duas condições	100% nas duas condições
Material	estudo-preliminar/	experimento/

O estudo preliminar bateu no teto da métrica, o que indicava que o sistema era fácil demais para diferenciar formatos de especificação. Junto com a mudança de escopo da atividade para microsserviços de logística, isso motivou o estudo principal, descrito a seguir.

2. Hipóteses

Seja C a conformidade de uma execução: a razão entre critérios de aceitação aprovados e o total de critérios (77).

H1 (principal). Microsserviços gerados a partir de um documento no modelo ERL têm conformidade acima de 85%.

- $H0_1$: a mediana de C na condição ERL é $\leq 0{,}85$.
- $H1_1$: a mediana de C na condição ERL é $> 0{,}85$.

H2 (secundária). A estrutura do modelo ERL aumenta a conformidade em relação a uma especificação narrativa com o mesmo conteúdo.

- $H0_2$: a conformidade da condição ERL não é estocasticamente maior que a da narrativa.
- $H1_2$: a conformidade da condição ERL é estocasticamente maior.

3. Método

3.1 Desenho

Experimento com um fator (formato da especificação), dois níveis e 30 execuções independentes por nível.

Variável	Tipo	Definição
Formato da especificação	Independente	**ERL**: ERS-LOG-001.md (≈ 51 KB). **Narrativa**: logistica-narrativa.md (≈ 12,5 KB)
Conformidade C	Dependente	CAs aprovados ÷ 77
Modelo, prompt, oráculo, barramento	Controladas	Iguais nas duas condições

3.2 Objeto do experimento

O módulo de logística é descrito em detalhe em `5-modulo-logistica.md`. Em resumo:

- **Transportadoras:** cadastro, com UFs atendidas, tarifas, prazo e peso máximo.
- **Cotação:** peso cubado, peso taxado, preço com adicional interestadual arredondado para cima, prazo em dias úteis, ordenação das opções e validade de 7 dias. Consulta Transportadoras por HTTP.
- **Envios:** criação com a **transportadora escolhida pelo cliente**, máquina de estados, devolução automática na 3ª falha e cancelamento. Consulta Cotação por HTTP e publica eventos.
- **Rastreamento** e **Notificações:** consomem eventos, com idempotência.
- **Barramento:** infraestrutura **fornecida pronta**, que não é gerada pela LLM.

No total são 16 requisitos funcionais, 15 regras de negócio, 5 requisitos não funcionais e 77 critérios de aceitação, distribuídos por serviço conforme a Seção 5.3.

Condição narrativa (controle). Contém todas as regras, rotas, campos, códigos de erro, ordens de validação, mensagens e valores do documento ERL, em prosa corrida, sem IDs, tabelas, glossário, cenário base, cenário integrado nem critérios de aceitação. A diferença entre as condições isola, portanto, a **estrutura e os exemplos**, e não a quantidade de informação.

3.3 Instrumento de medição (oráculo distribuído)

`experimento/testes/` contém um teste por critério de aceitação, nomeado pelo id do CA. O harness (`conftest.py`):

1. sobe o barramento e os cinco serviços com `uvicorn`, **em processos separados e portas livres**, passando as URLs por variável de ambiente;
2. testa tudo por HTTP, e verifica os eventos pelo registro do barramento;
3. reinicia o estado antes de cada teste pela rota `POST /admin/reset` e, se ela falhar, reinicia o processo do serviço. Assim, a falta de uma rota não derruba em cascata os demais critérios (ADR 0005).

Validação do oráculo (`executar.py validar`):

Verificação	Esperado	Obtido
Implementação de referência	77/77	77/77
Sistema vazio (5 arquivos sem código)	0/77	0/77
Mutante: peso cubado com divisor 5	< 77	73/77
Mutante: preço interestadual arredondado para baixo	< 77	76/77
Mutante: prazo em dias corridos	< 77	72/77
Mutante: cotação válida por 6 dias	< 77	75/77
Mutante: listagem ignora `somente_ativas`	< 77	76/77
Mutante: erro de validação no formato padrão do FastAPI	< 77	72/77
Mutante: devolução só na 4ª falha	< 77	74/77
Mutante: cotação consultada antes do pedido duplicado	< 77	76/77
Mutante: data e hora anteriores aceitas	< 77	76/77
Mutante: cancelamento depois da coleta	< 77	76/77
Mutante: rastreamento sem idempotência	< 77	76/77
Mutante: atrasado ignora estado terminal	< 77	76/77
Mutante: notificação também para COLETADO	< 77	73/77
Mutante: ordenação das opções sem desempate por `id`	< 77	**77/77 (não detectado)**

O oráculo aprova a referência, não dá crédito a um sistema vazio e detecta **13 dos 14 mutantes**. O mutante não detectado é **equivalente**: o serviço Cotação recebe as transportadoras do serviço Transportadoras já em ordem alfabética de `id` (RF-T02), e a ordenação do Python é estável, então retirar o desempate por `id` na Cotação não muda nenhuma resposta observável do sistema. Nenhum teste pela rede consegue distingui-lo da referência. A consequência prática é que o CA-C01-08 (desempate por `id`) verifica o comportamento do sistema como um todo, mas não garante que a Cotação implemente o desempate por conta própria. O escore de mutação entre os mutantes não equivalentes é de 13/13. Os testes não foram alterados depois da coleta.

Estabilidade do oráculo. No piloto, duas execuções apareceram com 88,3% e 90,9%. A análise de causa (fase F8) mostrou que os testes abriam uma conexão TCP nova a cada requisição e, com outra validação rodando em paralelo, esgotaram as portas efêmeras do Windows (erro `WinError 10048`). Depois da correção, com conexões reaproveitadas e avaliações em sequência, as mesmas 5 execuções deram 100%. Essa falha virou uma regra do processo: o oráculo também precisa ser estável, e as avaliações rodam uma de cada vez.

3.4 Procedimento

1. **Piloto:** 5 gerações ERL. Nenhum defeito de especificação foi encontrado, e o documento e o oráculo foram congelados.
2. **Coleta:** 60 gerações (30 ERL e 30 narrativas), em ordem sorteada com semente fixa (2026) e intercaladas entre as condições, por **agentes do Claude Code** (Claude Opus 5.5), um por

execução e sem histórico. Cada agente recebeu a instrução de ler **somente** o arquivo da sua especificação, cumprir o prompt fixo (`prompt.txt`) e gravar os cinco arquivos, sem executar nada.

3. **Interrupções:** o limite de uso da assinatura interrompeu 11 agentes no meio da geração. Cada um foi **retomado do ponto onde parou**, no mesmo contexto, e não reiniciado; cada execução continua sendo uma única geração. 7 execuções que ainda não tinham começado foram disparadas depois.
4. **Avaliação:** `executar.py registrar`, uma execução por vez, sem edição manual do código.

Regra de inclusão: toda execução entra na análise, e nenhuma foi descartada.

3.5 Análise estatística

Hipótese	Teste principal	Complementares	α
H1	Wilcoxon de postos sinalizados, uma amostra, contra 0,85, unilateral	Teste do sinal; t de Student (indefinido com variância zero); IC 95% por bootstrap	0,05
H2	Mann-Whitney U, unilateral	Correlação rank-biserial	0,05

---

4. Ambiente

Item	Valor
Modelo	Claude Opus 5.5, via agentes do Claude Code
Parâmetros de amostragem	Padrão do Claude Code, não controláveis
Execução dos testes	Python 3.14, FastAPI, uvicorn, httpx, pytest; Windows 11
Data	2026-10-21
Consumo	≈ 85 mil tokens por geração ERL e ≈ 65 mil por narrativa (≈ 4,9 milhões no total, com o piloto), dentro da assinatura; custo direto zero

---

5. Resultados

Dados brutos: `experimento/resultados/execucoes.csv`; código gerado em `experimento/resultados/<condição>/<execução>/servicos/`; tabelas em `resumo.md`.

5.1 Estatística descritiva

Condição	n	Média	DP	Mediana	Mín	Máx	IC 95%	Execuções ≥ 85%	Linhas de código (média, mín–máx)
ERL	30	100,0%	0,0%	100,0%	100,0%	100,0%	[100,0%; 100,0%]	30/30	790 (684–896)
Narrativa	30	100,0%	0,0%	100,0%	100,0%	100,0%	[100,0%; 100,0%]	30/30	763 (695–872)

Todas as 60 execuções subiram os cinco serviços e passaram nos 77 critérios.

5.2 Teste de H1

Teste	Estatística	p-valor	Decisão
Wilcoxon de postos sinalizados (principal)	W = 465,0	$2{,}16 \times 10^{-8}$	rejeita $H0_1$
Teste do sinal	30 de 30 acima de 0,85	$9{,}31 \times 10^{-10}$	rejeita $H0_1$
t de Student	indefinido (variância zero)	—	—

H1 é aceita. A conformidade dos microsserviços gerados a partir do documento ERL é maior que 85%, com $p < 0{,}001$, e o pior caso também foi 100%.

5.3 Conformidade por serviço

Serviço	CAs	ERL	Narrativa
Transportadoras	13	100%	100%
Cotação	14	100%	100%
Envios	29	100%	100%
Rastreamento	8	100%	100%
Notificações	7	100%	100%
Transversais (reset, RNF, cenário integrado)	6	100%	100%

5.4 Teste de H2

Teste	Estatística	p-valor	Tamanho de efeito	Decisão
Mann-Whitney U	indefinido (todas as execuções com o mesmo valor)	—	r = 0	não rejeita $H0_2$

H2 não pôde ser sustentada: as duas condições atingiram o teto, como no estudo preliminar.

5.5 Verificação de contaminação

Os agentes rodaram na máquina onde estão a referência e os testes. A semelhança textual entre o código gerado (os 5 arquivos concatenados) e a referência foi de 0,14 em média na condição ERL

(máximo 0,18) e de 0,18 na narrativa (máximo 0,23), bem abaixo do estudo preliminar (0,35). Os sistemas gerados têm entre 684 e 896 linhas, contra 507 da referência (linhas não vazias). Não há sinal de cópia.

---

6. Discussão

H1. O modelo ERL atingiu a meta com folga, agora num sistema distribuído: cinco processos que só se conhecem pela rede, com HTTP síncrono, eventos, idempotência, máquina de estados e regras numéricas com arredondamento e calendário. Nas 30 execuções, nenhum contrato entre serviços falhou: nenhum campo de evento com nome trocado, nenhuma rota divergente, nenhum erro de validação no formato padrão do framework.

H2 e o teto, pela segunda vez. Mesmo com um sistema cerca de 2,5 vezes maior em código e distribuído, a especificação narrativa também chegou a 100%. Os dois estudos apontam para a mesma conclusão: **para um modelo forte, o que garante a conformidade é o conteúdo da especificação, e não a sua forma**. A narrativa foi escrita depois do documento ERL e, por isso, já continha cada decisão que o processo força a tomar: a ordem dos erros, as fronteiras ("a cotação vale até `valida_ate`, inclusive"), os campos exatos de cada evento, a conversão do erro do FastAPI, as mensagens literais das notificações e o fato de a data da cotação não contar como dia útil.

Daí vem a principal lição do trabalho: **o valor do modelo ERL está em obrigar o analista a tomar e escrever essas decisões**. As seções de convenções gerais, catálogo de eventos, ordem de validação, cenário base e critérios de aceitação existem para que nenhuma delas fique implícita. O experimento mostra que, quando elas estão escritas, a LLM acerta em qualquer formato. O que ele **não** mede, porque o controle foi deliberadamente completo (ADR 0003), é quanto dessas decisões um analista escreveria sem o processo.

Por que o teto pode ser real, e não falta de sensibilidade. O oráculo detecta defeitos pequenos: uma ordem de validação invertida ou um arredondamento para baixo já reprovam critérios (Seção 3.3). A falta de variação, portanto, não vem de testes frouxos, e sim de o modelo não errar quando a especificação é completa.

---

7. Ameaças à validade

Tipo	Ameaça	Mitigação ou consequência
Interna	**Mesma família de modelo:** documento, referência e testes foram escritos com apoio do Claude, e o código foi gerado pelo Claude.	Declarada. Uma tentativa de replicar com o Gemini pela API gratuita falhou, porque os modelos disponíveis não tinham cota sem cartão. Replicar com outro fornecedor é a extensão mais importante.
Interna	**Isolamento dos agentes não auditável:** os agentes rodaram na máquina com a referência e os testes.	Semelhança com a referência baixa (0,14 a 0,18) e igual entre as condições (Seção 5.5).
Interna	**Interrupções por limite de uso:** 11 gerações pararam no meio.	Retomadas no mesmo contexto, sem reiniciar; todas as 60 completaram os 5 arquivos.
Interna	Parâmetros de amostragem não controlados.	Mesma configuração em todas as execuções. O script `executar.py gerar` permite repetir com parâmetros fixos via API.
Constructo	Um mutante equivalente (desempate por `id` na Cotação) não é detectável pela rede.	Declarado na Seção 3.3; o comportamento do sistema como um todo continua verificado pelo CA-C01-08. Um teste que injetasse transportadoras fora de ordem exigiria simular o serviço Transportadoras.
Interna	Instabilidade do ambiente de teste (portas no Windows).	Detectada no piloto, corrigida antes da coleta e avaliações feitas em sequência (Seção 3.3).
Constructo	Os 77 CAs não medem legibilidade, desempenho real nem tolerância a falhas.	A métrica mede aderência ao documento, que é o que a atividade pede.
Constructo	O controle narrativo tem todas as decisões do ERL, o que é raro num enunciado real.	Escolha deliberada, que torna H2 conservador e explica o resultado nulo.
Externa	Barramento síncrono e confiável; estado em memória; um domínio; uma linguagem.	Não generalizar para mensageria real, persistência ou outras stacks (limitações em `5-modulo-logistica.md`, Seção 7).
Conclusão	Variância zero.	Testes não paramétricos (Wilcoxon e sinal) continuam válidos. A conclusão vale para este modelo e este sistema.

---

8. Melhorias propostas

No processo e no artefato (incorporadas na versão 2):

1. **Seções próprias de microsserviços:** arquitetura com arquivo e variável de ambiente por serviço, mapa de comunicação, catálogo de eventos com campos exatos e convenções transversais escritas uma única vez.
2. **Testabilidade como atividade de análise (F2):** URLs por ambiente, datas na requisição, rota de reset e identificadores determinísticos. Sem isso não existe oráculo.
3. **Oráculo distribuído e estável (F5):** serviços em processos separados, testados pela rede, com conexões reaproveitadas, e avaliações em sequência.
4. **Cenário base nomeado (S0, Q-A, E-1):** deixa os 77 CAs curtos e sem ambiguidade.

5. **Testes por serviço, além dos de sistema:** o mutante equivalente da Seção 3.3 mostra que uma regra pode ficar coberta pelo comportamento de outro serviço. Testar cada serviço também isolado, com um dublê no lugar das dependências (ex.: Transportadoras devolvendo em ordem inversa), pegaria esse caso.

No experimento:

6. **Controle realista para H2:** pessoas de fora do grupo especificando o mesmo sistema sem o processo, a partir da mesma entrevista, para medir o efeito do **processo** e não só da forma.
7. **Outros modelos e fornecedores,** inclusive menores, nos quais a diferença entre formatos tende a aparecer.
8. **Infraestrutura real** (Kafka ou RabbitMQ, banco por serviço) e requisitos de falha (reentrega, ordem, *outbox*), para sair do teto.
9. **Medir o esforço** de preenchimento do modelo, para comparar custo e ganho.

---

9. Conclusão

O processo ERL e o modelo de documento para microsserviços atingiram o objetivo da atividade: em 30 gerações independentes, os cinco microsserviços de logística produzidos pelo Claude Opus 5.5 a partir do documento passaram nos 77 critérios de aceitação, com 100% de conformidade, acima da meta de 85% (Wilcoxon, $p = 2{,}2 \times 10^{-8}$). O resultado repete o do estudo preliminar num sistema maior e distribuído.

Nos dois estudos, uma especificação narrativa com o mesmo conteúdo também chegou a 100%. A lição para o processo é que, para uma LLM forte, a conformidade depende de as decisões estarem **tomadas e escritas**: contratos, ordem de erros, fronteiras e formatos. O modelo ERL é a ferramenta que obriga o analista a tomá-las e a não esquecer nenhuma. Medir esse efeito do processo, comparando com especificações escritas sem ele, é o próximo passo.

Referências

- ISO/IEC/IEEE 29148:2018. *Systems and software engineering — Life cycle processes — Requirements engineering.*
- SOMMERVILLE, I. *Engenharia de Software*. 10. ed. São Paulo: Pearson, 2018.
- NEWMAN, S. *Building Microservices*. 2. ed. Sebastopol: O'Reilly, 2021.
- RICHARDSON, C. *Microservices Patterns*. Shelter Island: Manning, 2018.
- WOHLIN, C. et al. *Experimentation in Software Engineering*. Berlin: Springer, 2012.
- JIA, Y.; HARMAN, M. An Analysis and Survey of the Development of Mutation Testing. *IEEE Transactions on Software Engineering*, v. 37, n. 5, 2011.
- NORTH, D. *Introducing BDD*, 2006.

Apêndices

- A. Modelo do documento: `2-modelo-documento-requisitos.md`
- B. Documento preenchido: `experimento/especificacoes/ERS-LOG-001.md`
- C. Especificação narrativa: `experimento/especificacoes/logistica-narrativa.md`
- D. Explicação do módulo: `5-modulo-logistica.md`
- E. Oráculo e harness: `experimento/testes/`; barramento: `experimento/infra/barramento.py`
- F. Dados brutos e código gerado: `experimento/resultados/`
- G. Estudo preliminar (biblioteca): `estudo-preliminar/relatorio-estudo-preliminar.md`